



Administrator Linux. Professional

Мониторинг производительности



Проверить, идет ли запись

Меня хорошо видно && слышно?



Преподаватель



Лавлинский Николай

Технический директор «Метод Лаб»

Более 15 лет в веб-разработке

Преподавал в ВУЗе более 10 лет
Более 5 лет в онлайн-образовании

Специализация: оптимизация производительности, ускорение сайтов и веб-приложений

https://t.me/methodlab_tg

<https://www.methodlab.ru/>

<https://rutube.ru/channel/24617406/>

<https://www.youtube.com/c/NickLavlinsky>

https://www.youtube.com/@site_support

<https://vkvideo.ru/@methodlab>

<https://habr.com/ru/users/Nickmob/>

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в Телеграм-чате



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом

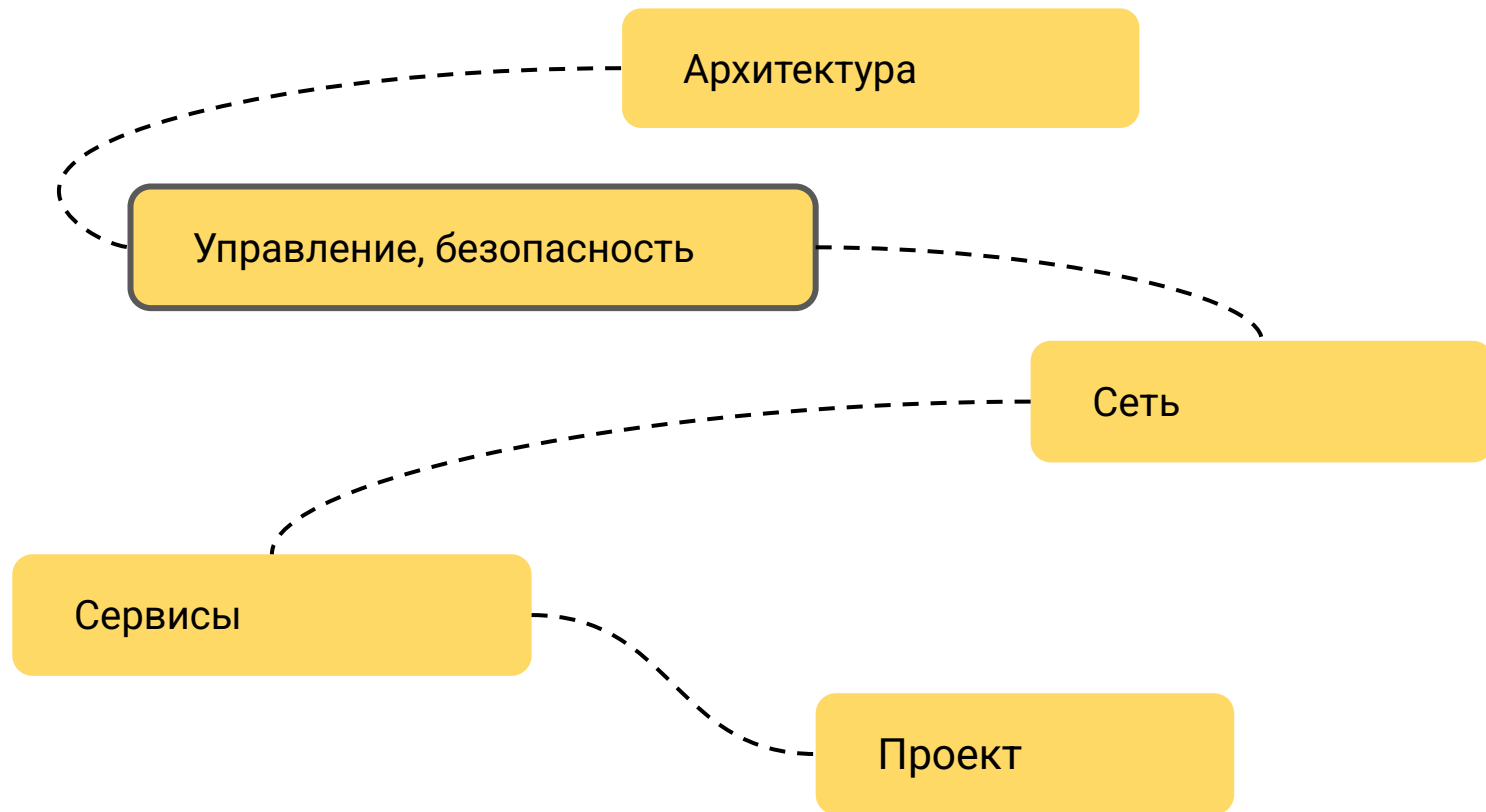


Документ



Ответьте себе или
задайте вопрос

Карта курса



Маршрут вебинара



Цели вебинара

После занятия вы сможете

1. Понимать методологии поиска проблемы производительности
2. Ориентироваться в утилитах для поиска проблем
3. Определять проблему, опираясь на факты

Смысл

Зачем вам это уметь

1. Оптимизация использования ресурсов
2. Увеличение стабильности работы
3. Отладка проблем в работе сервисов



Методологии поиска проблем

Зачем нужны формальные методы

- Определение наличия проблемы
- Поиск путей решения проблемы (алгоритм, средства, метрики)
- Анти-методологии как пример того, как не надо делать

https://www.brendangregg.com/Slides/EuroBSDcon2017_SystemMethodology/

Анти-методологии

Метод уличного фонаря

1. Выбрать инструмент, который:
 - Знаком
 - Нагуглен
 - Просто “кажется, что поможет”
2. Запустить
3. Получить результат
 - Посмотреть на очевидное
 - Погуглить, что могут значить полученные цифры



Метод пьяного админа

Рандомно менять настройки пока не поможет

Drunk Man *Anti*-Method

- ~~Drink~~ Tune things at random until the problem goes away



Обвини соседа

1. Найти систему или компонент вне вашей зоны ответственности
2. Высказать идею, в которой это является первопричиной
3. Переадресовать проблему команде, которая это поддерживает
4. Если не помогло: см. пункт 1

Blame Someone Else *Anti*-Method

1. Find a system or environment component you are not responsible for
2. Hypothesize that the issue is with that component
3. Redirect the issue to the responsible team
4. When proven wrong, go to 1



Настоящие методологии

Настоящие методологии

1. Problem statement method (описание проблемы)
2. Workload characterization method (описание нагрузки)
3. USE method
4. Off-CPU Analysis (анализ простоев)
5. CPY profile method
6. RTFM method (читай документацию)
7. Active benchmarking (тестирование производительности)
8. Static performance tuning (настройка системы)

Problem statement method

- Почему наблюдается проблема?
- Работала ли эта система лучше?
- Что поменялось в недавнее время? (Софт/Железо/Нагрузка)
- Можно ли охарактеризовать проблему с точки зрения задержек или времени выполнения?
- Кого аффецит проблема?
- Каков стек? Софт/Железо/Версии/Конфиги

Workload characterization method

- Что является причиной нагрузки (PID/UID/IP addr...)?
- Почему нагрузка возникла (stack trace/code path)?
- Природа нагрузки (IOPS, r/w, type)?
- Как нагрузка меняется с течением времени?

USE method

- Для каждого ресурса проверить:
 - **Utilization** (как загружен ресурс)
 - **Saturation** (длина в очереди или время в очереди)
 - **Errors** (логи ошибок)

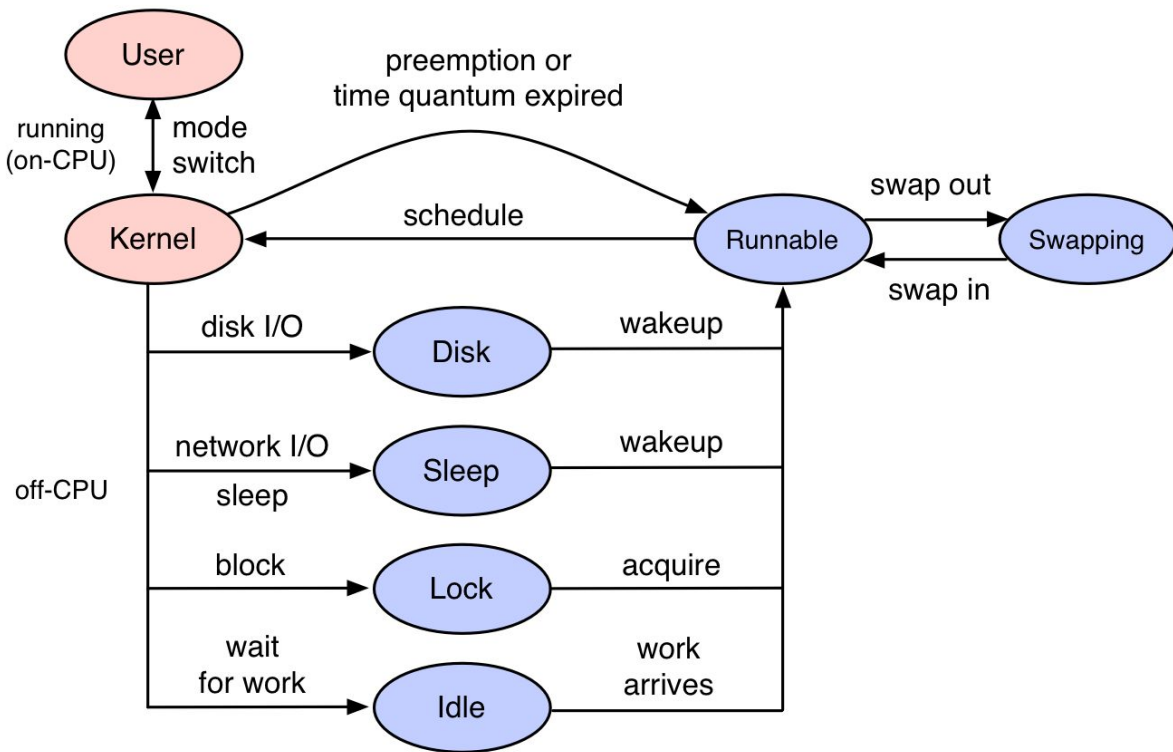
<https://www.brendangregg.com/USEmethod/use-linux.html>

RTFM

- Чтобы найти проблему нужно знать как оно все работает:
 - Man pages
 - Книги
 - Google
 - Коллеги
 - Source code
 - Эксперименты

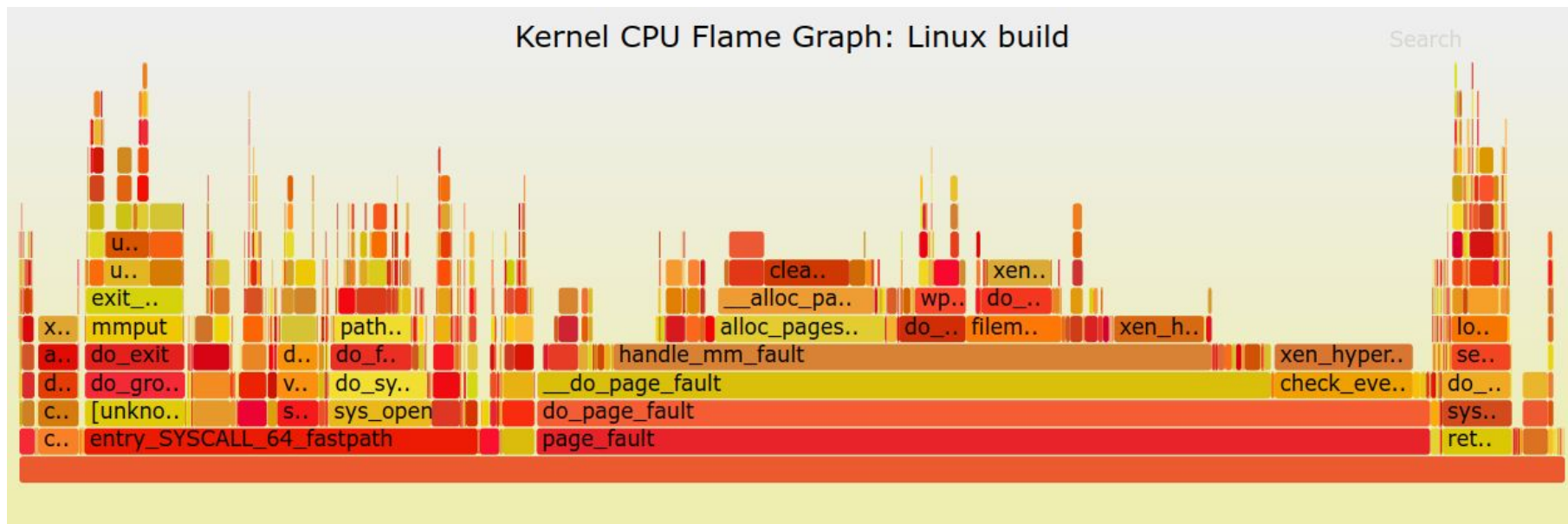
Off-CPU анализ

- Почему процесс не выполняется на ЦПУ?



Профилирование CPU

- perf record; perf report



Latency Numbers Every Programmer Should Know

■ 1 ns

■ L1 cache reference: 0.5 ns

■ Branch mispredict: 5 ns

■ L2 cache reference: 7 ns

■ Mutex lock/unlock: 25 ns

■ = 100 ns

■ Main memory reference: 100 ns

■ = 1 μ s

■ Compress 1 KB with Zippy: 3 μ s

■ = 10 μ s

■ Send 1 KB over 1 Gbps network: 10 μ s

■ SSD random read (1 Gb/s SSD): 150 μ s

■ Read 1 MB sequentially from memory: 250 μ s

■ Round trip in same datacenter: 500 μ s

■ = 1 ms

■ Read 1 MB sequentially from SSD: 1 ms

■ Disk seek: 10 ms

■ Read 1 MB sequentially from disk: 20 ms

■ Packet roundtrip CR to Netherlands: 150 ms

Source: <https://gist.github.com/2841832>

Вопросы?



Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет

Инструменты анализа

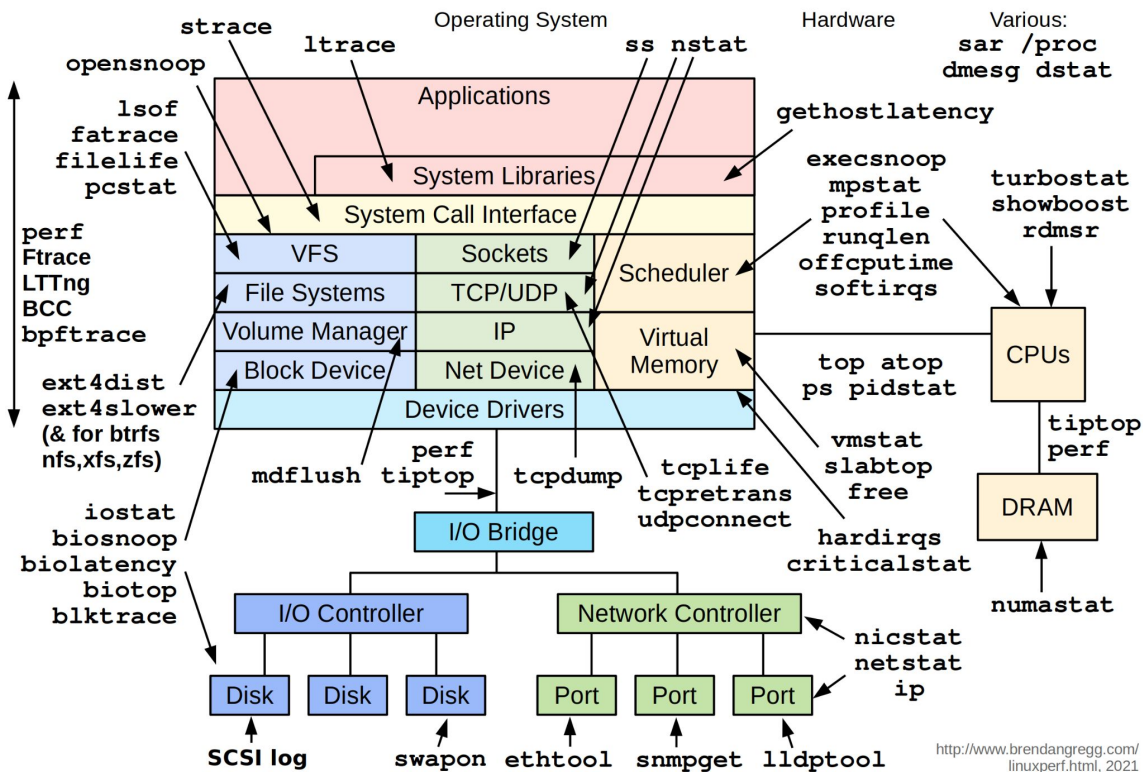
Типы инструментов

- **Observability** — посмотреть что происходит, обычно безопасно, но может быть overhead по ресурсам
- **Benchmarking** — проверка того, что система может выдержать. Можно что-нибудь поломать...
- **Tuning** — изменения настроек в системе
- **Static** — просмотр конфигов

https://www.brendangregg.com/Slides/LinuxConEU2014_LinuxPerfTools/

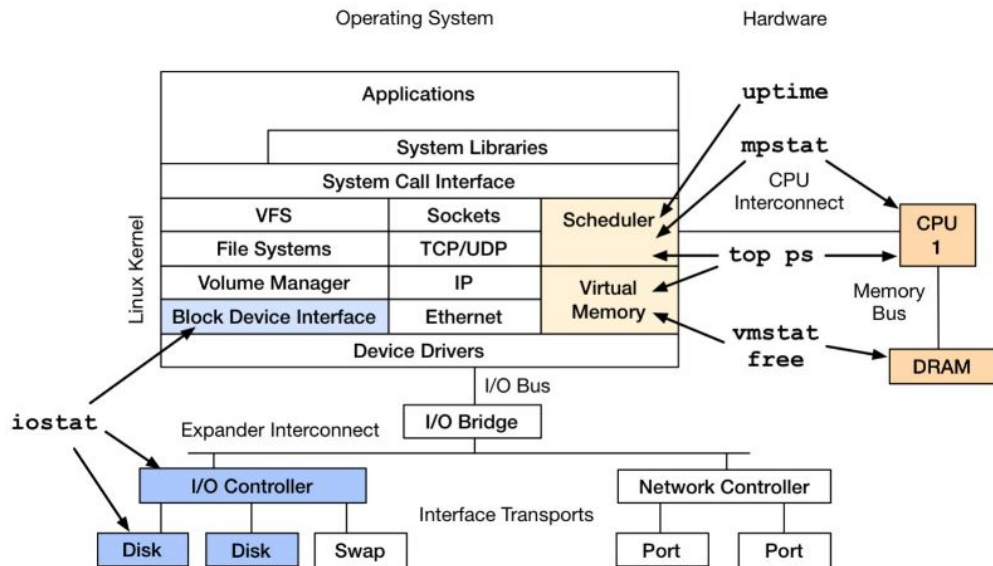
Observability tools

Linux Performance Observability Tools



Observability base

- uptime
- dmesg | tail; dmesg -T
- vmstat 1
- mpstat -P ALL 1
- pidstat 1
- iostat -xz 1
- free -m
- sar -n DEV 1
- sar -n TCP,ETCP 1
- top

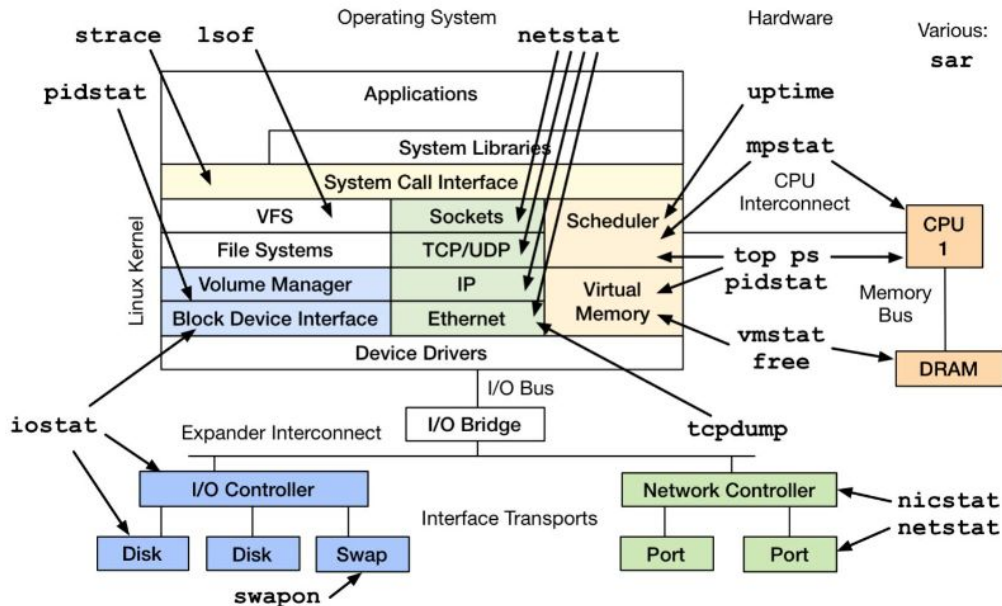


Brendan Gregg 2014

<https://www.youtube.com/watch?v=ZdVpKx6Wmc8>
<https://habr.com/ru/companies/vk/articles/335326/>
<https://phoenixnap.com/kb/swappiness>

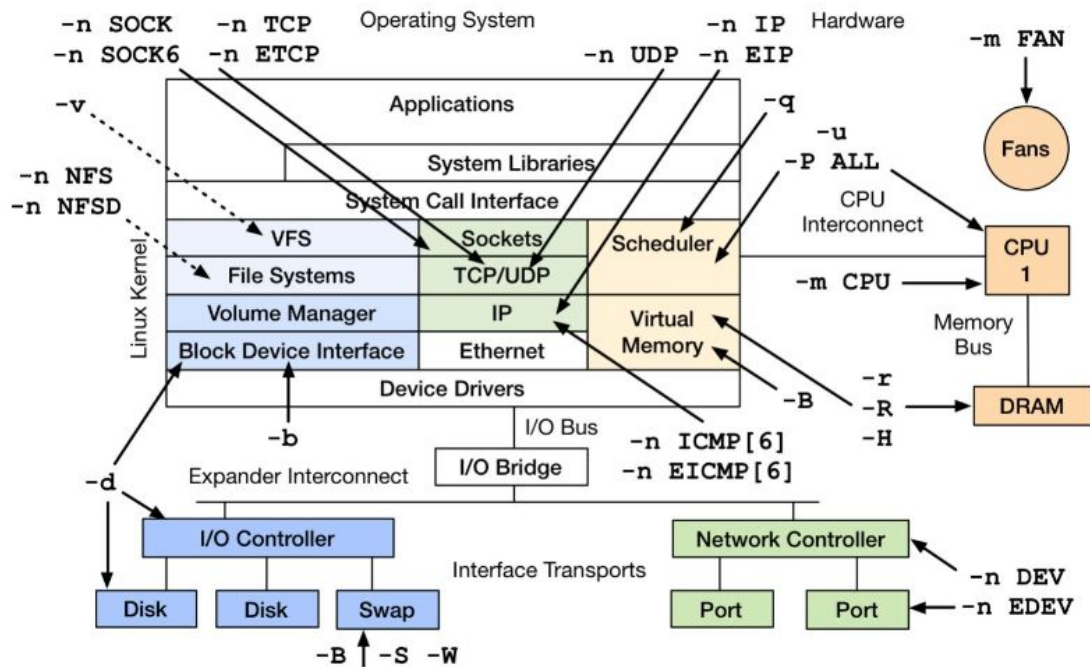
Observability Intermediate

- strace
- tcpdump
- netstat/ss
- nicstat
- pidstat
- swapon
- lsof
- sar



<http://www.brandangregg.com/linuxperf.html> 2014

Observability Intermediate SAR



Brendan Gregg 2014

Observability Advanced

- ltrace
- ethtool
- iotop
- blktrace
- slabtop
- /proc
- pcstat

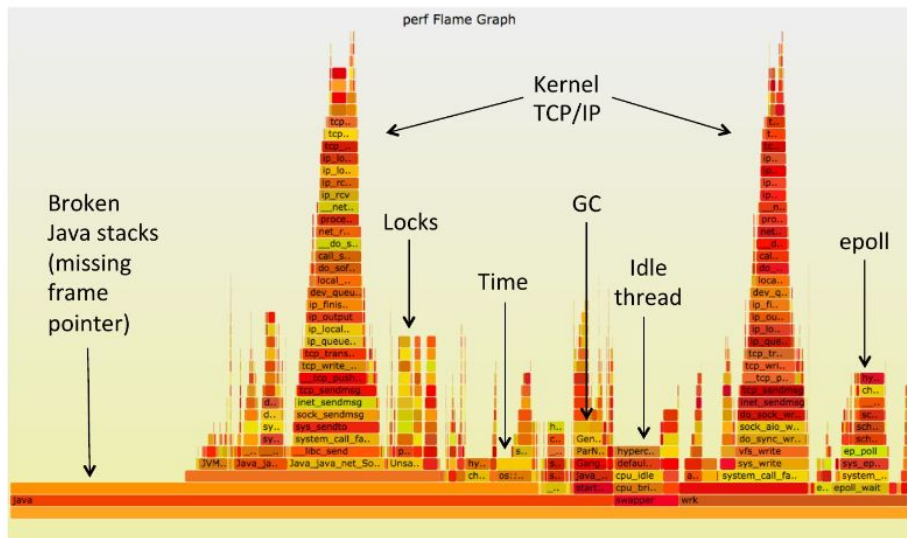
CPU performance counters:

- perf (perf_events), tiptop, rdmsr

Advanced tracers:

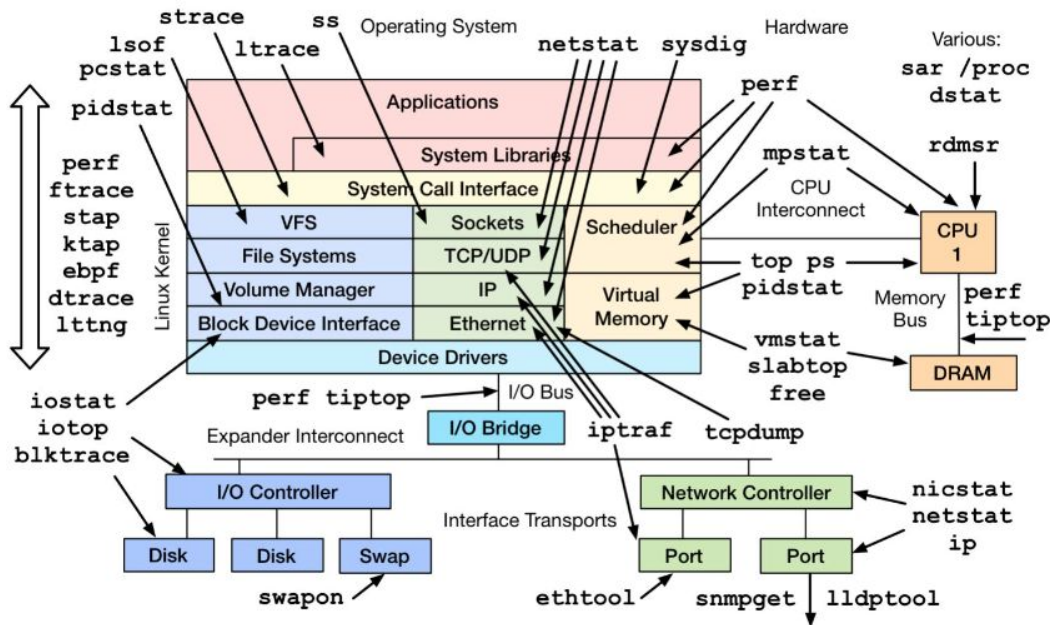
- perf (perf_events), ftrace, eBPF, dtrace

perf_events CPU Flame Graph



<https://www.brendangregg.com/perf.html>
<https://github.com/tobert/pcstat>

Observability Advanced



<http://www.brendangregg.com/linuxperf.html> 2014

Вопросы?



Ставим "+",
если вопросы есть



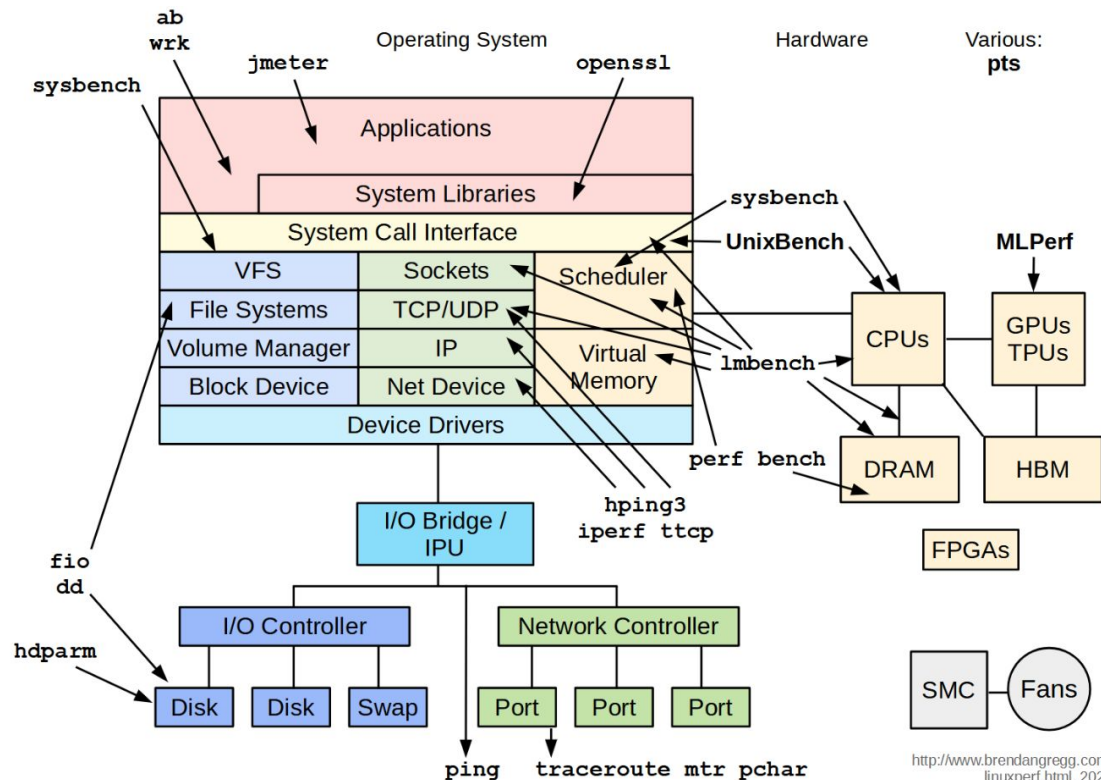
Ставим "-",
если вопросов нет

Инструменты тестирования Benchmark

Инструменты тестирования

- Все бенчмарки врут
- Универсальные: `unixbench/sysbench/perf bench`
- Диски: `dd/hdparm/fio`
- Приложение: `ab/wrk/jmeter/openssl/yandextank`
- Сеть: `ping/hping3/iperf/ttcp/mtr/pchar`

Инструменты тестирования

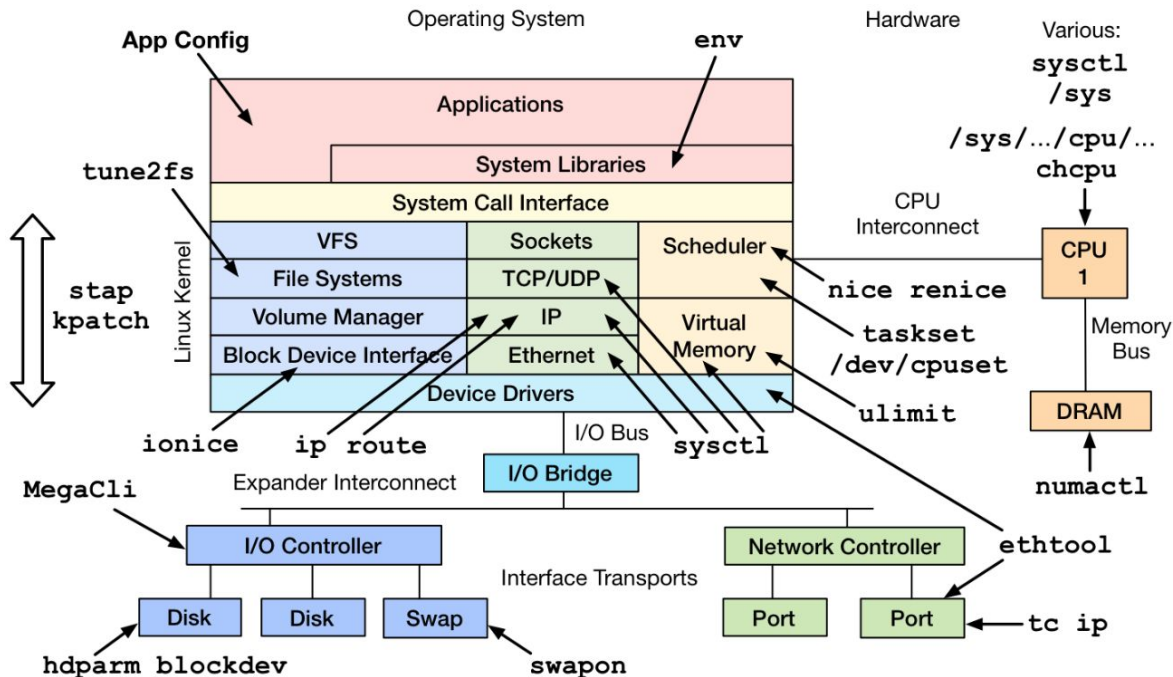


Инструменты настройки Tuning

Инструменты настройки

- Настройки ядра: `sysctl` или `/sys`
- Настройки приложения
- Планировщик, CPU/scheduler: `nice/renice/taskset/ulimit/chcpu`
- Диски, ввод-вывод: `tune2fs/ionice/hdparm/blockdev`
- Сеть: `ethtool/tc/ip/route`
- Динамические патчи: `stap/kpatch`

Инструменты настройки



<http://www.brendangregg.com/linuxperf.html> 2016

Вопросы?



Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет



Рефлексия

Тест

Вопросы для проверки

Какой метод предполагает проверку утилизации, насыщения и ошибок ресурсов (CPU, диск, сеть)?

- A. RTFM
- B. USE-метод
- C. Метод уличного фонаря
- D. Бенчмаркинг



Отправьте в чат верную букву

Вопросы для проверки

Какой метод предполагает проверку утилизации, насыщения и ошибок ресурсов (CPU, диск, сеть)?

A. RTFM

B. USE-метод

C. Метод уличного фонаря

D. Бенчмаркинг

Вопросы для проверки

Что такое Load Average в контексте мониторинга производительности?

- A. Среднее количество ошибок в системе
- B. Процент использования CPU
- C. Средняя длина очереди процессов, готовых к выполнению
- D. Количество активных сетевых соединений



Отправьте в чат верную букву

Вопросы для проверки

Что такое Load Average в контексте мониторинга производительности?

- A. Среднее количество ошибок в системе
- B. Процент использования CPU
- C. Средняя длина очереди процессов, готовых к выполнению**
- D. Количество активных сетевых соединений



Вопросы для проверки

Какая из характеристик присуща инструментам Observability (наблюдаемости)?

- A. Всегда приводят к изменению в системе.
- B. Обычно безопасно, но может создавать дополнительную нагрузку по ресурсам.
- C. Могут что-нибудь поломать.
- D. Показывают только статические конфигурации.



Отправьте в чат верную букву



Вопросы для проверки

Какая из характеристик присуща инструментам Observability (наблюдаемости)?

A. Всегда приводят к изменению в системе.

B. Обычно безопасно, но может создавать дополнительную нагрузку по ресурсам.

C. Могут что-нибудь поломать.

D. Показывают только статические конфигурации.

Вопросы для проверки

Какой инструмент используется для трассировки системных вызовов процесса?

- A. perf
- B. iotop
- C. sar
- D. strace



Отправьте в чат верную букву

Вопросы для проверки

Какой инструмент используется для трассировки системных вызовов процесса?

- A. perf
- B. iotop
- C. sar
- D. strace**

Вопросы для проверки

Что такое Off-CPU анализ?

- A. Анализ процессов, которые не используют CPU из-за ожидания I/O или блокировок
- B. Анализ процессов, которые используют 100% CPU
- C. Анализ сетевых пакетов
- D. Анализ ошибок в файловой системе



Отправьте в чат верную букву

Вопросы для проверки

Что такое Off-CPU анализ?

A. Анализ процессов, которые не используют CPU из-за ожидания I/O или блокировок

B. Анализ процессов, которые используют 100% CPU

C. Анализ сетевых пакетов

D. Анализ ошибок в файловой системе

Цели вебинара

Проверка достижения целей

1. Понимать методологии поиска проблемы производительности
2. Ориентироваться в утилитах для поиска проблем
3. Определять проблему, опираясь на факты

Рефлексия



Что было самым полезным на занятии?



Как будете применять на практике то, что узнали на вебинаре?

**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

Спасибо за внимание!

Приходите на следующие вебинары



Лавлинский Николай

Технический директор “Метод Лаб”

https://t.me/methodlab_tg
<https://www.methodlab.ru/>
<https://rutube.ru/channel/24617406/>
<https://www.youtube.com/c/NickLavlinsky>
https://www.youtube.com/@site_support
<https://vkvideo.ru/@methodlab>
<https://habr.com/ru/users/Nickmob/>

